



q_rnn_and_grad_sol.ipynb
File

You are an AI tutor and code reviewer for a Jupyter notebook that contains:

- a coding / ML problem,
- and the staff-provided solution (and possibly some tests or helper code).

Your goals:

- (1) Help me deeply understand the staff solution.
- (2) Actively quiz me with "what-if" code modification questions.
- (3) Critique the solution as a piece of software / ML engineering, not just as a correct answer.

I will upload a .ipynb file. Please work in the following phases.

PHASE 0 — ORIENT YOURSELF
#####

After reading the notebook:

- 1. Identify and briefly summarize:
 - The **problem statement** (in your own words).
 - Where the **main staff solution** lives (e.g., which cell / which function).
 - Any **supporting code** (helpers, tests, imports, plotting, etc.).
- 2. Output a short overview like:
 - Problem: [...]
 - Main solution entry point: [...]
 - Key helper functions: [...]
 - Tests / demo cells: [...]

If anything is ambiguous, say so instead of guessing.

PHASE 1 — EXPLAIN THE SOLUTION
#####

Assume I am a student who:

- knows basic Python,
- is familiar with lists/dicts/functions/loops,
- but may *not* know the specific algorithm or ML technique yet.

Do the following:

- 1. **High-level explanation (3–7 sentences)**
 - What is the core idea of the solution?
 - What algorithm / pattern does it implement (e.g., DP, DFS, greedy, training loop, etc.)?
- 2. **Guided code walkthrough**
 - Walk through the main solution function or cell **top-down**.
 - Group lines into logical chunks instead of line-by-line noise.
 - For each chunk, explain:
 - what it does,
 - why it's needed,
 - how it connects to the problem definition.
- 3. **Complexity / behavior**
 - State the time and space complexity if meaningful.
 - Mention any tradeoffs (e.g., readability vs performance, memory vs speed).

Keep this phase mostly explanatory, with occasional quick checks like:

> "Can you tell me in your own words what this loop is doing before I explain it?"

PHASE 2 — INTERACTIVE "WHAT IF?" QUESTIONS
#####

Now I want you to test and deepen my understanding by asking questions about code changes.

- 1. Ask **3–6 "what if" questions** such as:
 - "What do you think happens if we remove this condition?"
 - "If we change this list to a set, how does that affect correctness and complexity?"
 - "What if we change the iteration order here?"
 - "What if we initialize this variable differently?"
- 2. For each question:
 - Wait for my answer first.
 - Then:
 - say what *would* actually happen (behavior / correctness / performance),
 - and tie it back to the underlying concept or invariant.
- 3. Start with simpler changes (e.g., altering a constant or print) and

move toward more subtle ones (e.g., off-by-one edges, shared mutable state, ML training details).

If the code involves randomness or ML training:

- Ask at least one question about seeds, reproducibility, or how hyperparameters show up in the code.

PHASE 3 — SOFTWARE / ML ENGINEERING CRITIQUE
#####

Now critique the staff solution as if you were a senior software or ML engineer reviewing it.

1. **Style & Pythonic-ness**
 - Comment on:
 - naming (variables, functions),
 - function length and cohesion,
 - use of built-in functions / libraries,
 - avoiding repetition,
 - clarity vs cleverness.
 - Point out at least 3–5 concrete improvements, such as:
 - "This nested if could be simplified."
 - "This magic number should be a named constant."
 - "This could use a context manager / list comprehension / enumerate, etc."
2. **Structure & modularity**
 - Is the solution written as one big cell, or decomposed into testable functions?
 - Could it benefit from:
 - helper functions,
 - separating pure logic from I/O,
 - or separating data loading / config from core algorithm?
3. **Documentation & comments**
 - Are there docstrings or comments explaining key decisions?
 - Suggest where short comments or docstrings would significantly improve readability.
 - Optionally sketch an example docstring for the main function (using a standard style like NumPy or Google style).
4. **Testing & edge cases**
 - Does the notebook include tests or sample runs?
 - What edge cases might be missing?
 - Propose **3–5 simple test cases** that would be good to add (in plain English or as Python snippets).
5. **ML engineering (if applicable)**

- If the notebook includes any training/eval code:
- Comment on:
 - seeding / reproducibility,
 - separation of config (hyperparameters) from code,
 - logging / metrics,
 - handling of train/val/test,
 - use of vectorization vs for-loops.
- Suggest at least a couple of improvements (e.g., "move these constants to a config dict," "use a random seed," etc.).

6. **Concrete improved snippet(s)**

- Choose ONE small function or core block from the staff solution.
- Show:
 - "Original version" (copied and possibly shortened)
 - "Improved version" (your more Pythonic / maintainable rewrite)
- Explain in 3–5 bullets what changed and why it’s better (clarity, safety, extensibility, etc.).

IMPORTANT:

- Do NOT rewrite the entire notebook unless I explicitly ask.
- Focus on *illustrative* improvements that a student can learn from.

PHASE 4 — SUMMARY & ACTIONABLE NEXT STEPS
#####

Finish by giving me:

1. A **conceptual summary**:
 - 3–6 bullets summarizing:
 - the algorithm / method used,
 - the key ideas in the implementation,
 - and the main software-engineering lessons.
2. A **mini checklist** of things I could do on my own:
 - e.g.,
 - "Refactor the main function into two helpers,"
 - "Add tests for these edge cases,"
 - "Rewrite this loop using a more Pythonic construct."
3. Ask me:
 - "Which part of this notebook do you feel you understand the least?"
 - "Do you want to zoom in further on a particular function or cell?"

#####



STYLE & SAFETY

#####

- Be direct but encouraging; assume I am capable of understanding serious critique.
- Don't just say "this is bad"; explain *why* and how to improve it.
- If you are inferring intent that is not explicit in the notebook, make that clear:
 - > "I'm inferring that this function is meant to do X based on Y; if that's wrong, let's adjust."
- Avoid hallucinating details that don't appear in the notebook. If something is unclear, say so.

Thought for 7m 33s >

Here's **PHASE 0 — Orientation** based on your notebook.

1. What this notebook is about

- You're implementing **simple RNN and LSTM layers from scratch in PyTorch**, then
- Wrapping them in a **regression model** that predicts a running average of a 1-D sequence,
- Studying **gradient flow (exploding/vanishing)** through these networks using an interactive visualizer,
- Extending to **multi-layer RNN/LSTM** models, and
- **Training & comparing** 1-layer vs 2-layer RNNs/LSTMs on the synthetic sequence task.

2. Where the main stuff solutions live

I'll group the key pieces in the order they appear.

- **Single RNN layer (Problem 1.A)**
 - **Cell 6:** `class RNNLayer(nn.Module)`
 - `__init__` sets up a single linear layer that maps `[x_t, h_{t-1}] -> h_t`.
 - `forward` manually rolls the RNN over time, returning all hidden states and the last hidden state.
 - `store_h_for_grad` keeps a list of hidden states and calls `retain_grad()` for gradient visualization.
- **RNN regression wrapper (Problem 1.B)**
 - **Cell 10:** `class RecurrentRegressionModel(nn.Module)`
 - Wraps a recurrent net (`RNNLayer`, `LSTMLayer`, or multi-layer versions).
 - Adds a final linear layer `self.w` to map hidden states to output dimension.
 - `forward` runs the recurrent net, then applies `self.w` to every timestep.
- **Dataset & loss (Problem 1.C)**
 - **Cell 15:** `generate_batch(seq_len=10, batch_size=1)`

- Creates random 1-D sequences and targets equal to the **running average** of the sequence.
- **Cell 18:** `loss_fn(pred, y, last_timestep_only=False)`
 - Computes **MSE loss**, either over all timesteps or only the last one.
- **Gradient tools (Problem 1.D & 1.8b)**
 - **Cell 23:**
 - `biggest_eig_magnitude(matrix)` : largest eigenvalue magnitude helper.
 - `class GradientVisualizer` : builds plots showing **hidden activations and gradient norms** vs time, with sliders to rescale weights/biases.
 - **Cells 25 & 31:** use `GradientVisualizer` with `RNNLayer` and `LSTMLayer` .
- **Single-layer LSTM (Problem 1.H)**
 - **Cell 27:** `class LSTMLayer(nn.Module)`
 - `__init__` uses one `nn.Linear` to produce all four gates at once.
 - `forward` manually implements LSTM equations for `i`, `f`, `o`, `g` gates, cell state `c_t` , and hidden state `h_t` , storing the sequence of `h_t` .
- **Multi-layer RNN/LSTM (Problem 1.K.i)**
 - **Cell 34:**
 - `class RNN(nn.Module)` : stacks multiple `RNNLayer` objects in an `nn.ModuleList` , feeding layer output as next layer's input.
 - `class LSTM(nn.Module)` : similarly stacks `LSTMLayer` s and returns last layer's outputs and all last-step states.
- **Training + comparison (Problem 1.K.ii)**
 - **Cell 38:** `train(model, optimizer, num_batches, last_timestep_only, ...)`
 - Standard PyTorch training loop over mini-batches from `generate_batch` .
 - **Cell 39:** `train_all(hidden_size, lr, num_batches, last_timestep_only)`
 - Builds 4 models: {RNN, LSTM} × {1-layer, 2-layer}, trains each, and plots:
 - Loss curves,
 - Example input vs target vs predictions.
 - **Cell 40:** Sets `hidden_size` , `lr` , etc., seeds PyTorch, and runs `train_all` twice:
 - once predicting **all timesteps**,
 - once predicting **only the last timestep**.

3. Supporting code / tests

- **Imports & setup**
 - **Cell 3:** Standard imports (`torch` , `nn` , `optim` , `matplotlib` , `numpy` , `ipywidgets` , etc.).
 - **Cell 2 & 4:** Colab / `ipymp1` setup; not core to the algorithms.
- **Unit-style sanity checks**
 - **Cell 8:** Tests `RNNLayer` :
 - Checks shape, deterministic behavior under fixed weights, and parameter count.
 - **Cell 12:** Tests `RecurrentRegressionModel` :
 - Checks parameter count and forward output shapes.

- Cell 20: Tests `loss_fn` .
- Cell 29: Tests `LSTMLayer` .
- Cell 36: Tests multi-layer `RNN` and `LSTM` .
- Visualization
 - Cell 16: Plots sample `x` and running averages `y` to show the task (all timesteps vs last timestep only).
 - `GradientVisualizer` calls create interactive plots with sliders.

That’s Phase 0.

Before we dive into **Phase 1 (explaining the solution in detail)**:

- Does this summary match how *you* understood the notebook so far?
- Is there a particular part you’re most interested in starting with (e.g., `RNNLayer` , `LSTMLayer` , or the training loop), or should I start from `RNNLayer` and go top-down from there?

...

Yes the summary matches the notebook. Do top down

Thought for 2m 56s >

Nice, let’s go top-down starting from the core building block: the **RNN layer**.

PHASE 1 — High-level idea

At a high level, the staff solution is doing this:

- It builds **recurrent layers** (RNN and LSTM) that process a sequence one timestep at a time, maintaining a hidden state.
- These layers are then wrapped into a **regression model** that maps each hidden state to a scalar prediction (the running average of the input).
- For RNN: the hidden state update is
$$h_t = \text{nonlinearity}(W[x_t; h_{t-1}]).$$
- For LSTM: the layer uses **gates** (input, forget, output, candidate) and a cell state c_t to control information flow and help avoid vanishing/exploding gradients.
- Multi-layer versions simply **stack** these layers, feeding the output sequence of one layer as the input sequence to the next.

We’ll start with `RNNLayer` , since everything else (multi-layer RNN, regression model, training loop) sits on top of that.

`RNNLayer` — constructor



python

Copy code

```
class RNNLayer(nn.Module):
    def __init__(self, input_size, hidden_size, nonlinearity=th.tanh):
        ...
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.nonlinearity = nonlinearity

        self.w = nn.Linear(input_size + hidden_size, hidden_size, bias=True)
```

What this does:

- Stores the dimensions:
 - `input_size` : number of features per timestep (here it's 1).
 - `hidden_size` : dimension of the hidden state h_t .
- Stores the nonlinearity (default `torch.tanh`).
- Creates **one fully connected layer** `self.w` that maps the concatenation of:
 - the current input $x_t \in \mathbb{R}^{\text{input_size}}$,
 - and the previous hidden state $h_{t-1} \in \mathbb{R}^{\text{hidden_size}}$,to the next hidden state $h_t \in \mathbb{R}^{\text{hidden_size}}$.

So conceptually:

$$[z_t] = [x_t; h_{t-1}] \in \mathbb{R}^{\text{input_size} + \text{hidden_size}}, \quad h_t = \text{nonlinearity}(W z_t + b)$$

This is a **vanilla RNN cell** implemented with a single `nn.Linear` .

RNNLayer — forward pass

python

Copy code

```
def forward(self, x):
    """
    Inputs:
    - x: (B, seq_len, input_size)
    Returns:
    - all_h: (B, seq_len, hidden_size)
    - last_h: (B, hidden_size)
    """
    h_list = []
    b = len(x)
    h_i = th.zeros((b, self.hidden_size)).float()
    for i in range(x.shape[1]):
        in_value = th.cat([x[:, i], h_i], dim=1)
        h_i = self.nonlinearity(self.w(in_value))
        h_list.append(h_i)
        print(h_i.size())
    last_h = h_i

    self.store_h_for_grad(h_list)
```



```
all_h = th.stack(h_list, dim=1)
return all_h, last_h
```

Let's break this into chunks.

1. Input and initial hidden state

python

Copy code

```
h_list = [] # to store [h_1, ..., h_T]
b = len(x) # batch size
h_i = th.zeros((b, self.hidden_size)).float()
```

- `x` has shape `(B, T, input_size)`.
- They create:
 - `h_i` as the initial hidden state h_0 , all zeros, shape `(B, hidden_size)`.
 - `h_list` to store each hidden state $h_1, \dots, h_T$ as the RNN rolls through the sequence.

This matches the usual RNN idea: start with $h_0 = 0$.

2. Rolling over time

python

Copy code

```
for i in range(x.shape[1]):
    in_value = th.cat([x[:, i], h_i], dim=1)
    h_i = self.nonlinearity(self.w(in_value))
    h_list.append(h_i)
    print(h_i.size())
last_h = h_i
```

For each timestep `i`:

1. `x[:, i]` picks out the inputs at timestep `i` for all batch elements: shape `(B, input_size)`.
2. `th.cat([x[:, i], h_i], dim=1)` concatenates current input and previous hidden state along feature dimension $\rightarrow$ shape `(B, input_size + hidden_size)`.
3. `self.w(in_value)` applies the linear transformation, giving a raw next hidden vector `(B, hidden_size)`.
4. `self.nonlinearity(...)` (tanh by default) squashes it, giving the next hidden state `h_i`.
5. They append `h_i` to `h_list` so they can later:
 - return the whole sequence of hidden states,
 - inspect gradients through time.
6. `last_h` is just the final `h_i` after the loop.

The `print(h_i.size())` is a debugging check to make sure shapes are as expected; it's not conceptually part of the algorithm.



Connection to the problem:

For this homework, the RNN's job is to digest the input sequence and keep in its hidden state the information needed to approximate the **running average**. Later, a separate linear layer turns each h_t into a prediction $\hat{y}_t$.

3. Stacking and returning

python

Copy code

```
self.store_h_for_grad(h_list)
all_h = th.stack(h_list, dim=1)
return all_h, last_h
```

- `self.store_h_for_grad(h_list)` :
 - Calls `retain_grad()` on each h_t so you can later inspect `h_t.grad` after backprop. This is specifically for the **gradient visualization** part of the assignment.
- `th.stack(h_list, dim=1)` :
 - Takes a list of tensors, each $(B, \text{hidden_size})$, and stacks them along a new time dimension $\rightarrow (B, T, \text{hidden_size})$.
- Returns:
 - `all_h` : all hidden states over time,
 - `last_h` : the final hidden state h_T .

This is exactly what higher-level code (your regression model, multi-layer models, visualizer) needs.

4. Helper for gradients

python

Copy code

```
def store_h_for_grad(self, h_list):
    for h in h_list:
        h.retain_grad()
    self.h_list = h_list
```

- This is a pure "supporting for analysis" function.
- Normally, intermediate tensors' gradients aren't kept around individually.
- By calling `retain_grad()`, you can later inspect how gradients flow back through time (vanishing/exploding behavior).

Complexity for `RNNLayer`

Assume:

- Batch size = B
- Sequence length = T



- Input size = I
- Hidden size = H

Per timestep:

- One linear layer on a vector of size $(I + H)$ to produce size $H \rightarrow \sim O(H(I + H))$.
- Over all timesteps $\rightarrow$ **time complexity** roughly $O(B \cdot T \cdot H(I + H))$.
- **Space complexity:**
 - Activations: storing all `h_t` $\rightarrow O(B \cdot T \cdot H)$.
 - Plus extra for autograd graphs, but big-O wise that's the main factor.

Quick check for you

Let's pause here just on `RNNLayer` before moving on to LSTM and the multi-layer wrappers.

Suppose `x` has shape `(B=32, seq_len=10, input_size=1)` and `hidden_size=16`. Inside the loop, what shape do you expect `in_value` to have, and what shape do you expect `h_i` to have right after `h_i = self.nonlinearity(self.w(in_value))` ?

Tell me your guess in terms of actual numbers (like `(32, something)`). Once you answer, we'll continue top-down with the `LSTMLayer`, stacked `RNN / LSTM` classes, and the regression model + training loop.



in_value: (32, 17)
h_i: (32, 16)

Thought for 2m 32s >

Yup, perfect:

- `in_value`: `(32, 1 + 16) = (32, 17)`
- `h_i`: `(32, 16)`

Let's keep going **top-down** through the rest of the solution.

1. `RecurrentRegressionModel` — wrapping the RNN/LSTM

python Copy code

```
class RecurrentRegressionModel(nn.Module):
    def __init__(self, recurrent_net, output_dim=1):
        """
        Initialize a simple RNN regression model
        """
```

```

super().__init__()
self.recurrent_net = recurrent_net
self.output_dim = output_dim
...

```

Conceptually:

- `recurrent_net` is **any** recurrent module:
 - `RNNLayer`, `LSTMLayer`, `RNN` (multi-layer), or `LSTM` (multi-layer).
- `output_dim` is how many numbers we want to predict per timestep (here it's 1: running average scalar).

Inside `__init__` (hidden by `...` in this copy, but implied by the tests):

- It creates a **linear head** roughly like:

python

 Copy code

```
self.w = nn.Linear(recurrent_net.hidden_size, output_dim)
```

This maps each hidden state `ht` to a prediction $\hat{y}_t$.

forward of RecurrentRegressionModel

python

 Copy code

```

def forward(self, x):
    """
    Inputs:
    - x: (B, seq_len, input_size)
    Returns:
    - out: (B, seq_len, output_dim)
    - all_h: (B, seq_len, hidden_size)
    """
    all_h, _ = self.recurrent_net(x)
    out = self.w(all_h)
    return out, all_h

```

Chunk-by-chunk:

1. `all_h, _ = self.recurrent_net(x)`
 - Calls the underlying RNN/LSTM.
 - By the contract of `RNNLayer` / `RNN` / `LSTMLayer` / `LSTM`, this returns:
 - `all_h`: (B, seq_len, hidden_size) — hidden states at every timestep.
 - `_`: last state(s) (RNN: last hidden; LSTM: (last hidden, last cell)); not used here.
2. `out = self.w(all_h)`
 - Because `all_h` is (B, T, H), and `self.w` is a linear layer mapping H -> `output_dim`, PyTorch automatically treats it as a linear map over the last dimension.
 - Result: (B, T, output_dim).



So this class is: **recurrent feature extractor + linear regressor at each timestep.**

Quick check

Given:

- `input_size = 1,`
- `hidden_size = 8,`
- `output_dim = 1,`
- `recurrent_net = RNNLayer(1, 8),`

roughly how many **learnable parameters** do you expect in `RecurrentRegressionModel` ?
(You don't need the exact number, just describe what pieces contribute to it.)

2. Dataset generation — `generate_batch`

python

Copy code

```
def generate_batch(seq_len=10, batch_size=1):
    data = th.randn(size=(batch_size, seq_len, 1))
    sums = th.cumsum(data, dim=1)
    div = (th.arange(seq_len) + 1).unsqueeze(0).unsqueeze(2)
    target = sums / div
    return data, target
```

- `data` : random normal inputs, shape `(B, T, 1)`.
- `sums = cumsum(data, dim=1)` :
 - `sums[:, t]` is the sum of `data[:, 0..t]`.
- `div = (arange(seq_len) + 1)` :
 - `[1, 2, 3, ..., T]`, then reshaped to `(1, T, 1)` so it broadcasts.
- `target = sums / div` :
 - For each `t`, target is:

$$y_t = \frac{1}{t + 1} \sum_{k=0}^t x_k$$

- i.e. the **running average**.

The RNN/LSTM must learn to approximate this running average from scratch.

3. Loss function — `loss_fn`

python

Copy code

```
def loss_fn(pred, y, last_timestep_only=False):
    """
    pred: (batch, seq_len, 1)
    y:    (batch, seq_len, 1)
    """
    if last_timestep_only:
```



```

    pred, y = pred[:, -1], y[:, -1]
    loss = nn.MSELoss()(pred, y)
    return loss

```

- If `last_timestep_only=False`:
 - Compute MSE between full sequences `pred` and `y`.
- If `last_timestep_only=True`:
 - Slice to the last timestep:
 - `pred[:, -1]` and `y[:, -1]` → shape `(batch, 1)` (or `(batch,)` depending on how PyTorch broadcasts).
 - Compute MSE only on **final outputs**.

This switch lets you study how different training signals (all timesteps vs only last) affect gradient flow and performance.

Quick check

If `last_timestep_only=True`, do you think training becomes **easier** or **harder** for very long sequences (say `seq_len = 100`)?

Just your intuition: what happens to gradients that must travel all the way back from `t=99` to `t=0`?

4. Gradient visualizer — GradientVisualizer

At a high level (without going line-by-line through all the plotting code):

- `biggest_eig_magnitude(matrix)`:
 - Computes the magnitude of the **largest eigenvalue** of a square matrix.
 - This is related to how much repeated multiplication by this matrix can expand or shrink vectors — relevant to exploding/vanishing.
- `class GradientVisualizer`:
 - Takes:
 - an RNN module (`RNNLayer` or `LSTMLayer`),
 - and `last_timestep_only` flag (same as in `loss_fn`).
 - In `update_plots`:
 - Rescales the RNN weights and biases using slider values.
 - Runs a forward pass on a fixed input.
 - Runs a backward pass on the loss.
 - Uses `self.rnn.h_list` (remember `retain_grad()`) to collect gradients $\frac{\partial L}{\partial h_t}$.
 - Plots gradient norm vs timestep on a log scale.
 - The widget created by `create_visualization()` lets you see how:
 - scaling weights up/down,
 - or scaling biases
 - changes the **eigenvalues** of the recurrent weight matrix and the **gradient decay/growth** across time.

Conceptually: it builds intuition that **eigenvalues** $> 1 \rightarrow$ **exploding**, much $< 1 \rightarrow$ **vanishing**.



5. LSTMLayer — single-layer LSTM

The cell in this notebook is partially hidden (... in the version I can see), but based on the interface and tests:

python Copy code

```
class LSTMLayer(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.w = nn.Linear(input_size + hidden_size, hidden_size * 4)
        ...
```

Key idea:

- We again concatenate `[x_t, h_{t-1}]`.
- We apply **one linear layer** `self.w` that outputs a vector of size `4 * hidden_size`.
- Then we **split** this into four gate vectors:
 - input gate `i_t`,
 - forget gate `f_t`,
 - output gate `o_t`,
 - candidate `g_t` (sometimes called `\tilde{c}_t`).
- Standard LSTM equations:

$$\begin{aligned} [i_t, f_t, o_t, g_t] &= W[x_t; h_{t-1}] + b \\ i_t &= \sigma(i_t), \quad f_t = \sigma(f_t), \quad o_t = \sigma(o_t), \quad g_t = \tanh(g_t) \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

- The forward function:
 - Initializes `h_0` and `c_0` to zeros.
 - Loops over timesteps, applying the above equations.
 - Builds `h_list` exactly like `RNNLayer`.
 - Returns:
 - `all_h`: (B, T, hidden_size) from all timesteps,
 - `(h_last, c_last)`: both (B, hidden_size).

So compared to `RNNLayer`, `LSTMLayer` has:

- more parameters (since `hidden_size * 4` outputs),
- more internal state (`c_t`),
- but better control over gradient flow.

Complexity:

Still $O(B \cdot T \cdot H(I + H))$ time per forward; constants are higher due to more gates.

Quick check

In your own words:
Why does having a separate **cell state** `c_t` + gates help with the vanishing gradient problem compared to a plain `tanh(W[x_t; h_{t-1}])` RNN?
(Doesn't need to be formal—just your intuition about “remembering” and “forgetting”).

6. Multi-layer RNN and LSTM

Multi-layer RNN

python

Copy code

```
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers):
        super().__init__()
        assert num_layers >= 1
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        ...
        layers = [RNNLayer(input_size, hidden_size)]
        for i in range(num_layers - 1):
            layers.append(RNNLayer(hidden_size, hidden_size))
        self.layers = nn.ModuleList(layers)
```

- Builds a **stack** of RNN layers:
 - First layer: `input_size` → `hidden_size`.
 - Higher layers: `hidden_size` → `hidden_size`.

Forward:

python

Copy code

```
def forward(self, x):
    """
    Multilayer RNN forward pass
    """
    h_list = []
    for i, layer in enumerate(self.layers):
        all_h, last_h = layer(x)
        x = all_h
        h_list.append(last_h)
    last_layer_h = all_h
    last_step_h = th.stack(h_list)
    return last_layer_h, last_step_h
```

Interpretation:

- `x` initially: `(B, T, input_size)`.
- For each layer:
 - `all_h` becomes the new input for the next layer (same shape but different features).

- `last_h` is the final hidden state of that layer `(B, H)`.
- After the loop:
 - `last_layer_h`: `(B, T, H)` from the top-most layer.
 - `last_step_h`: stacked final hidden states from all layers $\rightarrow$ shape `(num_layers, B, H)`.

So this is a **vertical depth** in addition to time depth.

Multi-layer LSTM

Similar pattern:

python

 Copy code

```
class LSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers):
        super().__init__()
        assert num_layers >= 1
        layers = [LSTMLayer(input_size, hidden_size)]
        for i in range(num_layers - 1):
            layers.append(LSTMLayer(hidden_size, hidden_size))
        self.layers = nn.ModuleList(layers)
```

Forward:

python

 Copy code

```
def forward(self, x, hc0=None):
    """
    Multilayer LSTM forward pass
    """
    h_list, c_list = [], []
    for i, layer in enumerate(self.layers):
        all_h, (last_h, last_c) = layer(x)
        x = all_h
        h_list.append(last_h)
        c_list.append(last_c)
    last_layer_h = all_h
    last_step_h = th.stack(h_list)
    last_step_c = th.stack(c_list)
    return last_layer_h, (last_step_h, last_step_c)
```

- Same stacking idea, but now we keep:
 - `last_step_h`: final hidden for each layer,
 - `last_step_c`: final cell for each layer.

7. Training loop — `train` and `train_all`

`train`

python

```

def train(model, optimizer, num_batches, last_timestep_only, seq_len=10, batch_si
    model
    model.train()

    losses = []
    from tqdm import tqdm
    t = tqdm(range(0, num_batches))
    for i in t:
        data, labels = generate_batch(seq_len=seq_len, batch_size=batch_size)
        pred, h = model(data)
        loss = loss_fn(pred, labels, last_timestep_only)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if i % 100 == 0:
            t.set_description(f"Batch: {i} Loss: {np.mean(losses[-10:])}")
    return losses

```

Key points:

- `model.train()` puts it into training mode (important if you had dropout, batchnorm, etc.).
- For each batch:
 - Sample new (data, labels) from `generate_batch`.
 - Forward: `pred, h = model(data)`.
 - Compute loss via `loss_fn`.
 - Backprop: `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`.
- Store `loss.item()` for plotting.

Note: the stray `model` line at the top is a no-op; likely a leftover.

train_all

python

 Copy code

```

def train_all(hidden_size, lr, num_batches, last_timestep_only):
    input_size = 1
    rnn_1_layer = RecurrentRegressionModel(RNN(input_size, hidden_size, 1))
    lstm_1_layer = RecurrentRegressionModel(LSTM(input_size, hidden_size, 1))
    rnn_2_layer = RecurrentRegressionModel(RNN(input_size, hidden_size, 2))
    lstm_2_layer = RecurrentRegressionModel(LSTM(input_size, hidden_size, 2))
    models = [rnn_1_layer, lstm_1_layer, rnn_2_layer, lstm_2_layer]
    model_names = ['rnn_1_layer', 'lstm_1_layer', 'rnn_2_layer', 'lstm_2_layer']

    losses = []
    for model in models:
        optimizer = optim.Adam(model.parameters(), lr=lr)

```



```

    loss = train(model, optimizer, num_batches, last_timestep_only)
    losses.append(loss)

# visualize the results
...
return models, losses

```

- Builds 4 models:
 - RNN vs LSTM × 1-layer vs 2-layers.
- Trains each with Adam.
- Collects their loss curves and then:
 - Plots loss vs batch index for each model.
 - Plots example predictions vs true running average for visual comparison.

This is the **experimental harness** to compare architectures and depth.

Complexity for the whole training

Per batch:

- `generate_batch` : $O(B \cdot T)$.
- `model` forward + backward:
 $O(B \cdot T \cdot H(I+H) \cdot \text{num_layers})$.
- So per model: $O(\text{num_batches} \cdot B \cdot T \cdot H(I+H) \cdot \text{num_layers})$.

Tradeoffs:

- More layers and larger hidden sizes improve capacity but:
 - Increase compute and memory.
 - Can make gradient flow more fragile (especially for vanilla RNNs).

One last check for Phase 1

If you look at `train_all`, they use the **same** learning rate `lr` and number of batches for all 4 models.

Why might that be *unfair* or at least *not fully informative* when comparing 1-layer RNN vs 2-layer LSTM performance?

Give me your thoughts (even if they're half-formed), and then we can move into **Phase 2** — “**what if**” **questions** where you reason about code changes and their effects.



A 2-layer LSTM usually needs a different learning rate and more training steps to show its advantages. Using the same lr and same number of batches tends to favor the



simpler 1 layer RNN, so the comparison isn't really fair or fully informative.

Exactly 👍 — same hyperparams across very different models mostly tells you “with *this* setup, these models behave like this,” not “this model is inherently better.”

PHASE 2 — “What if?” Questions

Let’s play with some code-perturbation thought experiments to deepen your understanding.

Question 1 — Removing the nonlinearity

Right now, `RNNLayer` does:

python

Copy code

```
in_value = th.cat([x[:, i], h_i], dim=1)
h_i = self.nonlinearity(self.w(in_value)) # usually tanh
```

Suppose we change the constructor to:

python

Copy code

```
self.nonlinearity = lambda x: x # identity, i.e., no nonlinearity
```

So the layer becomes a purely **linear** recurrent system.

What do you think would happen, qualitatively, to:

- 1. The **expressive power** of the RNN on the running-average task (can it still learn it? will it be more/less limited)?
- 2. The **gradient behavior** you’d see in the `GradientVisualizer` (e.g., exploding/vanishing more or less likely, and why, thinking in terms of eigenvalues)?

Give me your best guess first, then I’ll tell you what would actually happen and why.

📄 👍 💬 ↺ ...

Loses expressive power

Yep, that’s the key first piece: it definitely loses expressive power 🧠

A purely linear RNN is much more limited in what kinds of sequence mappings it can learn (it’s basically doing a fancy linear transform over time).

Let’s finish that thought experiment before I fill in details:



Still on Question 1:

If we remove the nonlinearity (so the RNN is purely linear), what do you expect to happen to the **gradient behavior** over many timesteps?

- “Do you think **exploding/vanishing gradients** would become more likely, less likely, or about the same — and why, in terms of repeatedly multiplying by the same weight matrix and its eigenvalues?”

Give me your intuition first, then I’ll tell you what actually tends to happen and why, and we’ll move on to the next “what if.”



Exploding gradients

Thought for 27s >

Nice intuition 💡 — let’s unpack it a bit and then I’ll give you the next “what if.”

What actually happens in Question 1 (no nonlinearity)

If we remove the nonlinearity, the recurrence becomes:

$$h_t = Wh_{t-1} + Ux_t + b$$

When you backprop through time, gradients involve repeated multiplication by $W^\top$:

$$\frac{\partial L}{\partial h_k} \propto (W^\top)^{T-k} \dots$$

- If the **largest eigenvalue magnitude** (spectral radius) of W is > 1 , then powers of W blow up → **exploding gradients**.
- If it’s < 1 , powers of W shrink to zero → **vanishing gradients**.

So your answer “exploding gradients” is half the story:

👉 *both* exploding and vanishing are more “pure” and directly tied to the eigenvalues of W in the linear case.

With **tanh**:

- Activations are bounded in $[-1, 1]$, and
- The derivative $\tanh'(z) \leq 1$ adds extra multiplicative factors in $(0, 1)$, which tends to **push toward vanishing**, but also somewhat “damps” explosions from very large linear responses.
- In practice, nonlinearities make the behavior more complicated (and often more stable in terms of activations), but you still see vanishing/exploding issues.

In the **GradientVisualizer**, with a linear RNN you’d see gradient curves much more tightly match what the spectral radius predicts (almost pure geometric growth/decay).

Question 2 — Changing the stacking dimension

Right now in `RNNLayer.forward` we have:

pythonCopy code

```
all_h = th.stack(h_list, dim=1) # (B, T, H)
```

Suppose we instead wrote:

pythonCopy code

```
all_h = th.stack(h_list, dim=0) # (T, B, H)
```

and didn't change anything else in the code (e.g., `RecurrentRegressionModel`, `loss_fn`, etc.).

What do you think would happen?

- Would the code still run, or would something break?
- If it runs, where would the **shape mismatch** show up?
- If it breaks, at what point in the pipeline do you expect the error (in the regression model, the loss function, etc.)?

Give me your best guess, then we'll talk through exactly how that dimensionality change ripples through the rest of the solution.



The code would run but there would be a shape mismatch in computing loss

Thought for 16s >

Nice, you're basically on the right track 🧙

What actually happens with `dim=0` in `th.stack`

Original:

pythonCopy code

```
all_h = th.stack(h_list, dim=1) # (B, T, H)
```

Your change:

pythonCopy code

```
all_h = th.stack(h_list, dim=0) # (T, B, H)
```

- Each `h_i` in `h_list` is `(B, H)`.
- With `dim=1`, you get `(B, T, H)` → **batch, time, hidden**.

- With `dim=0`, you get `(T, B, H)` → **time, batch, hidden**.

Now, how does this flow?

- `RecurrentRegressionModel` does:

python

 Copy code

```
all_h, _ = self.recurrent_net(x)
out = self.w(all_h)
```

- `self.w` is a linear map over the last dim, so `(T, B, H) -> (T, B, 1)` is fine.
- But `loss_fn` expects:

python

 Copy code

```
pred: (batch, seq_len, 1)
y:    (batch, seq_len, 1)
```

and we pass:

- `pred` with shape `(T, B, 1)`
- `y` from `generate_batch` with shape `(B, T, 1)`

PyTorch broadcasting can't reconcile `(T, B, 1)` vs `(B, T, 1)` (the `T` and `B` are in different positions and both > 1), so you get a **runtime shape error at loss computation**.

So your answer "there would be a shape mismatch in computing loss" is spot on for the **single-layer** case 🍌

For multi-layer RNN/LSTM, it's even sneakier:

- The first layer would output `(T, B, H)` and set `x = all_h`.
- The second `RNNLayer` assumes `x` is `(B, T, input_size)` (batch in dim 0, time in dim 1), so it silently treats:
 - `batch = T`,
 - `seq_len = B`.
- That doesn't immediately crash, but the semantics are completely wrong and **later** shapes stop matching expectations.

So the big lesson: **time vs batch dimension conventions must stay consistent across all modules**.

Next "what if" (Question 3)

Right now, the RNN starts with a **zero** hidden state:

python

 Copy code

```
h_i = th.zeros((b, self.hidden_size)).float()
```

What if we changed this to a random initialization each forward pass, like:

python

```
h_i = th.randn((b, self.hidden_size))
```

and left everything else the same.

What do you think this would do to:

- 1. **Training stability** and convergence (loss curves)?
- 2. The **interpretability** of the `GradientVisualizer` plots across different runs?

Give me your intuition first, then I'll walk through what would actually happen.

    ...

loss curves would be noisier, convergence slower and less stable

Thought for 26s >

Yeah, your intuition is good here 👍

What actually happens with random `h_0`

Original:

```
python
h_i = th.zeros((b, self.hidden_size)).float()
```

 Copy code

What-if version:

```
python
h_i = th.randn((b, self.hidden_size))
```

 Copy code

1. Training stability / convergence

- Every forward pass now starts from a **different hidden state**, even if:
 - the model weights are the same, and
 - the input batch is the same.
- That means:
 - The model can't rely on a consistent "starting memory."
 - Gradients have extra noise coming from the randomness in `h_0`, not just from data.
- So as you said:
 - Loss curves become **noisier** (more fluctuation from batch to batch).
 - Convergence is often **slower** and sometimes **less stable**, because the model has to "fight against" this random initial condition.
 - A smart optimizer may learn to **suppress dependence on early timesteps** (e.g., by shrinking the recurrent connection), which ironically makes long-term memory

worse.

For a simple task like the running average, it'll probably still learn, but with more variance across runs.

2. GradientVisualizer interpretability

- The visualizer is meant to show **how gradient norms change across time** as you scale weights/biases.
- If `h_0` is random each call:
 - The **input state** to the recurrence changes every time,
 - So gradient patterns can jitter from one run to the next,
 - Making it harder to attribute changes in the plot purely to "I turned this slider up/down."
- With `h_0 = 0`, you get a **clean, fixed reference**: any change in gradient profiles is directly attributable to weights/biases, not initial state randomness.

So your answer (noisier, slower, less stable) is exactly how I'd summarize it 🙌

Question 4 — Reproducibility / seeds

In the training cell they do something like:

python Copy code

```
th.manual_seed(42)
np.random.seed(42)
```

before building models and calling `train_all(...)`.

What if we removed those seeding lines entirely, and left everything else the same?

What do you think would happen to:

1. The **loss curves** across different runs of the notebook?
2. The **relative comparison** between RNN vs LSTM, 1-layer vs 2-layer — would it become more or less reliable?

Give me your best guess, and then I'll explain what actually changes and why this matters for good ML practice.

📄 👍 💬 ↺ ...

Loss curves would change slightly. Less reliable comparison

Exactly – that's the right high-level takeaway 🙌

What actually happens without seeding

If we remove:



python

Copy code

```
th.manual_seed(42)
np.random.seed(42)
```

then:

1. Loss curves across runs

- Each time you run the notebook:
 - Model weights initialize differently.
 - `generate_batch` uses different random inputs.
- So:
 - The *shape* of the loss curve (e.g., “generally going down”) stays similar.
 - But the **exact values** and even which model is slightly ahead at a given batch can change.
- In some unlucky cases, one run might show LSTM “worse” than RNN just by random chance, and another run the opposite.

2. Reliability of comparisons

- You lose **reproducibility**: you can’t re-run an experiment and get the same curves to debug or verify.
- Comparisons like “2-layer LSTM converges faster than 1-layer RNN” become **less trustworthy** if they’re based on a single, unseeded run.
- Good practice: either
 - fix seeds to compare implementations, **or**
 - run multiple seeds and compare **distributions** of performance.

So your answer (“loss curves change slightly; less reliable comparison”) is exactly the right intuition 🎯

Question 5 — Last-timestep-only vs all-timesteps loss

Right now, `loss_fn` has:

python

Copy code

```
if last_timestep_only:
    pred, y = pred[:, -1], y[:, -1]
loss = nn.MSELoss()(pred, y)
```

and in `train_all` they run **two experiments**:

- one with `last_timestep_only=False`,
- one with `last_timestep_only=True`.

What if we accidentally flipped the flag when calling `train_all`, so that:

- the **intended** “all-timesteps” experiment is actually training with `last_timestep_only=True`, and
- the “last-timestep” experiment is actually training with all timesteps?

Based on your understanding of gradient flow:

1. For **very long sequences** (say `seq_len = 100`), which experiment would now show *worse* learning for the **earlier** timesteps?
2. How would the **GradientVisualizer** curves differ between these two (where do you expect gradients to be smallest)?

Give me your best reasoning first, then we'll dissect what's really happening and move on to critique (Phase 3).

    ...

"all-timesteps" would now look worse on early timesteps.
On to the next phase

Thought for 1m 56s >

Great, let's move into critique mode 🐙 (in a helpful way).

I'll *briefly* close the loop on that last question, then do **Phase 3** and **Phase 4**.

Quick wrap on the last-timestep flag (1–2 sentences)

If you accidentally flip `last_timestep_only` between experiments, you end up **supervising a different set of timesteps than you think**. For long sequences, the "supposed" all-timestep experiment will secretly push gradients mostly from the last step, so earlier timesteps will learn worse, and your comparison becomes misleading.

Okay, onto the review.

PHASE 3 — Software / ML engineering critique

1. Style & Pythonic-ness

Good things:

- Clear, short class names: `RNNLayer`, `LSTMLayer`, `RNN`, `LSTM`, `RecurrentRegressionModel`, `GradientVisualizer`.
- Most functions have docstrings describing inputs/outputs.
- Tests are written in plain Python with `assert` and small examples — very readable.

Concrete improvements:

1. **Unused expression in** `train`:

python

 Copy code

```
def train(...):  
    model  
    model.train()
```

- `model` on its own line does nothing — probably a leftover.
- **Fix:** delete that line.

2. Imports inside functions:

python

Copy code

```
from tqdm import tqdm
```

- Done **inside** `train`.
- More conventional to put imports at the **top of the notebook** so they're visible in one place and only executed once.
- Also helps static analysis tools and linters.

3. Magic numbers:

- Examples:
 - In `train`: `if i % 100 == 0:` (logging frequency).
 - In plotting code: `seq_len` is implicitly assumed to be 10 in a few places.
- Better to expose these as named parameters (`log_every=100` , `seq_len` from function args) or constants at the top.

4. Docstring typo / truncation:

- In `loss_fn` docstring, `lat` instead of `last`.
- Not a big deal, but small inconsistencies add friction when skimming.

5. Variable naming:

- `data` , `labels` are clear.
- In some places, `h` vs `last_h` vs `all_h` could be clarified with comments/docstring emphasizing shapes:
 - E.g., "`all_h` : (B, T, H) hidden states per timestep; `last_h` : (B, H) final timestep."

6. Progress bar naming:

- `t = tqdm(range(0, num_batches))` → something like `progress = tqdm(...)` or `pbar = tqdm(...)` is more self-documenting.

Overall: the style is pretty decent, but cleaning up imports, magic numbers, and a few stray lines would make it feel more "production-ish."

2. Structure & modularity

Positives:

- Core logic is already split into:
 - Layer classes (`RNNLayer` , `LSTMLayer`),
 - Multi-layer wrappers (`RNN` , `LSTM`),
 - Model wrapper (`RecurrentRegressionModel`),
 - Utilities (`generate_batch` , `loss_fn`),
 - Training (`train` , `train_all`),
 - Visualization (`GradientVisualizer`).
- This is **much better** than one giant "everything in one cell" notebook.

Where it could be more modular:



1. `train_all` mixes training and plotting.

- It:
 - builds models,
 - trains them,
 - plots loss curves,
 - plots predictions.
- More modular design:
 - `models, losses = train_all_models(...)`
 - `plot_losses(model_names, losses)`
 - `plot_predictions(models, model_names, ...)`
- This would make it easier to:
 - Run training once,
 - Rerun only plotting with different subsets / styles.

2. `GradientVisualizer` mixes math and UI.

- Same class both:
 - computes eigenvalues, runs forward/backward, collects gradients,
 - manages `ipywidgets` sliders and plotting.
- A slightly more "library-like" structure:
 - A pure "backend" function: `run_gradient_experiment(rnn, scale_w, scale_b, ...)`.
 - A UI wrapper that calls that function and plugs into widgets.

3. Separation of "model definition" vs "experiment config."

- Hyperparameters (`hidden_size`, `lr`, `num_batches`, `seq_len`) are currently passed around as positional arguments or are implicit in cells.
- It'd be cleaner to have a small config dict, e.g.:

python

Copy code

```
config = {  
    "hidden_size": 10,  
    "lr": 1e-3,  
    "num_batches": 5000,  
    "seq_len": 10,  
    "batch_size": 32,  
}
```

and then pass `config` into `train_all`.

3. Documentation & comments

What's good:

- Most functions have docstrings describing:
 - inputs,
 - outputs,
 - basic intent.

- Inline comments in tests clarify what’s being checked (param counts, shapes, deterministic behavior).

What could be improved:

1. Shape-centric comments in recurrent layers.

- For `RNNLayer.forward` and `LSTMLayer.forward` , a short comment like:

python

Copy code

```
# x: (B, T, input_size)
# return all_h: (B, T, hidden_size), last_h: (B, hidden_size)
```

right above `return` helps a lot.

2. Clarify training/eval expectations.

- In `train` , a docstring could explicitly say:
 - “Trains on synthetic running-average data generated by `generate_batch` .”
 - “Assumes model returns `(preds, hidden)` .”

3. Explain the purpose of `GradientVisualizer` in its docstring.

- Right now it’s more input/outputs oriented.
- A one-liner like:

“Helps visualize exploding/vanishing gradients in RNNs as we rescale parameters.”

Example docstring (Google-style) for `train` :

python

Copy code

```
def train(model, optimizer, num_batches, last_timestep_only, seq_len=10, batch_si
    """Train a recurrent regression model on the running-average task.

    Args:
        model: nn.Module that maps x -> (preds, hidden_states).
        optimizer: torch.optim.Optimizer instance for updating model parameters.
        num_batches: Number of training mini-batches to run.
        last_timestep_only: If True, compute loss only at final timestep.
        seq_len: Length of each input sequence.
        batch_size: Number of sequences per batch.

    Returns:
        List of scalar loss values (one per batch).
    """
```

4. Testing & edge cases

What’s already there:

- `RNNLayer` tests:
 - Check param count.
 - Check output shapes and numeric values under fixed weights.

- **RecurrentRegressionModel test:**
 - Checks parameter counts, shapes, and some deterministic behavior.
- Likely similar tests for `LSTMLayer`, multi-layer `RNN` / `LSTM`, and `loss_fn`.

These are great “sanity/contract” tests.

Extra tests that would be helpful:

1. Sequence length = 1.

python

 Copy code

```
x, y = generate_batch(seq_len=1, batch_size=4)
preds, _ = model(x)
assert preds.shape == (4, 1, 1)
```

Ensures indexing `[:, -1]` in `loss_fn` and time loop boundaries handle minimal sequences.

2. Larger batch size + longer seq_len.

python

 Copy code

```
x, y = generate_batch(seq_len=50, batch_size=64)
preds, _ = model(x)
assert preds.shape == (64, 50, 1)
```

Catches any accidental assumptions about fixed batch/seq sizes.

3. Check that `loss_fn` behaves correctly in both modes.

python

 Copy code

```
pred = th.ones(2, 3, 1)
y = th.zeros(2, 3, 1)
assert loss_fn(pred, y, last_timestep_only=False).item() == 1.0
assert loss_fn(pred, y, last_timestep_only=True).item() == 1.0
```

(This is simplified, but ensures slicing doesn’t change scaling.)

4. Gradient flow sanity check.

- For a tiny model and short sequence, ensure `loss.backward()` runs without error and some gradients are non-zero.

5. LSTM-specific test for cell state.

- E.g., ensure that `LSTMLayer` returns `(all_h, (last_h, last_c))` with correct shapes and that the shapes match the spec.

5. ML engineering aspects

Good practices already present:

- Random seeds set with `th.manual_seed` and `np.random.seed` → reproducible experiments.
- Clean separation of:

- data generation: `generate_batch`
- model definition: RNN/LSTM classes
- training: `train` , `train_all`
- Use of Adam optimizer, which is robust for this kind of toy setup.

Improvements from an ML engineering lens:

1. Config + logging instead of hard-coded values.

- Right now, you have to scroll around to change `hidden_size` , `lr` , `num_batches` , etc.
- A small `config` dict or dataclass at the top would make experiments easier to tweak.
- Basic logging to a dict or CSV would make it easier to compare runs.

2. Device handling (CPU vs GPU).

- All tensors are created with `th.randn` / `th.zeros` on default device.
- For more realistic scenarios:
 - Decide on `device = th.device("cuda" if th.cuda.is_available() else "cpu")` .
 - Move models and data to that device.

3. Gradient clipping (RNN-specific).

- To mitigate exploding gradients, it’s common to add:

python

Copy code

```
th.nn.utils.clip_grad_norm_(model.parameters(), max_norm)
```

before `optimizer.step()` .

4. Train/validation separation.

- Here the task is synthetic and stationary, so it’s fine.
- But as a pattern, it would be good to show:
 - training loop,
 - occasional validation on a held-out batch distribution.

6. Concrete improved snippet

Let’s pick the `train` function — it’s central and has some easy wins.

Original version (from the notebook)

python

Copy code

```
def train(model, optimizer, num_batches, last_timestep_only, seq_len=10, batch_si
model
model.train()

losses = []
from tqdm import tqdm
t = tqdm(range(0, num_batches))
for i in t:
    data, labels = generate_batch(seq_len=seq_len, batch_size=batch_size)
```



```

    pred, h = model(data)
    loss = loss_fn(pred, labels, last_timestep_only)
    losses.append(loss.item())

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if i % 100 == 0:
        t.set_description(f"Batch: {i} Loss: {np.mean(losses[-10:])}")
    return losses

```

Improved version (same behavior, cleaner style)

python

 Copy code

```

# Ideally at the top of the notebook:
from tqdm import tqdm

def train(
    model,
    optimizer,
    num_batches,
    last_timestep_only,
    seq_len=10,
    batch_size=32,
    log_every=100,
):
    """Train a recurrent regression model on the running-average task.

    Args:
        model: nn.Module that returns (preds, hidden) from input x.
        optimizer: torch.optim.Optimizer for updating model parameters.
        num_batches: Number of training mini-batches.
        last_timestep_only: If True, compute loss only at final timestep.
        seq_len: Length of input sequences.
        batch_size: Number of sequences per batch.
        log_every: How often to update the progress bar description.

    Returns:
        List of scalar loss values (one per batch).
    """
    model.train()
    losses = []

    progress = tqdm(range(num_batches))
    for batch_idx in progress:
        data, labels = generate_batch(seq_len=seq_len, batch_size=batch_size)
        preds, _ = model(data)
        loss = loss_fn(preds, labels, last_timestep_only)
        losses.append(loss.item())

        optimizer.zero_grad()

```



```

        loss.backward()
        optimizer.step()

    if batch_idx % log_every == 0:
        recent_mean = np.mean(losses[-10:])
        progress.set_description(f"Batch {batch_idx} Loss {recent_mean:.4f}")

    return losses

```

What improved and why:

- Removed the unused `model` line → avoids confusion.
- Imports `tqdm` at module level → more standard.
- Added a **docstring** explaining behavior and arguments.
- Used a more descriptive variable `progress` and `batch_idx` → clearer intent.
- Introduced `log_every` parameter → no magic `100` in the function body.
- Cleaned up formatting and consistent naming (`preds` vs `pred`).

You could apply similar cleanup to `train_all` and the plotting section.

PHASE 4 — Summary & actionable next steps

1. Conceptual summary

- The notebook implements **vanilla RNNs and LSTMs from scratch**, plus multi-layer variants, then uses them for a **running-average regression task**.
- `RNNLayer` and `LSTMLayer` process sequences one timestep at a time, returning both the full sequence of hidden states (B, T, H) and a final state.
- `RecurrentRegressionModel` wraps any recurrent net and adds a **linear output head** at each timestep.
- `loss_fn` supports **all-timestep** vs **last-timestep-only** training, which strongly affects gradient flow and difficulty for long sequences.
- The codebase is fairly modular already but can be improved by:
 - separating training vs plotting,
 - centralizing hyperparameters,
 - adding more explicit docs, tests, and ML-engineering niceties (seeds, configs, device, clipping).

2. Mini checklist for you

If you want to level this up as a learning exercise, here are concrete things you could try:

- ☐ Refactor `train_all` into:
 - a pure training function (`train_all_models`) and
 - separate plotting helpers (`plot_losses` , `plot_predictions`).
- ☐ Add a small `config` dict at the top of the notebook for `hidden_size` , `lr` , `num_batches` , `seq_len` , `batch_size` , `log_every` .
- ☐ Add 3–4 unit tests:
 - one for `loss_fn` in both modes,

- one for `LSTMLayer` shapes,
- one for `RNN / LSTM` with `num_layers=2`,
- one for `seq_len=1`.
- ☐ Clean up style issues:
 - remove unused `model` line,
 - move `from tqdm import tqdm` to the top,
 - fix small typos in docstrings.
- ☐ (Optional) Add gradient clipping in `train` and see how the gradient visualizer's behavior changes when you push weights to "exploding" regimes.